Claude Multi-Account Fine Tuning

llmctl Integration — User Guide

1. Detection
2. Switching
3. LAN-exposure warning
4. Context-size warning
5. Troubleshooting
5a. Model validation gate (§11.4.277) and Vision QA (§11.4.278)
6. What this integration does not do
7. Diagrams

llmctl Integration — User Guide

llmctl is a separate, sibling project that runs local LLM models — each as its own independent server process on a fixed, catalog-assigned port — with an OpenAI-compatible `/v1/models` / `/v1/chat/completions` API. `claude-providers` automatically recognizes whichever llmctl profiles you currently have **running**, exposes each as a launchable Claude Code and Kimi Code alias, and keeps that recognition current on every sync. This is the deep-dive reference; see `./quickstart.md` for the short version and `./FAQ.md` for common questions.

Nothing here modifies llmctl itself — see §6.

---

1. Detection

`detect_llmctl_records()` (in `scripts/claude-providers.sh`) runs on every `claude-providers sync` and is the single source of truth for which llmctl profiles currently exist as aliases:

- It asks llmctl itself for its current catalog (`llmctl plan --json`) to learn each profile's name and **resolved port** — never a

hardcoded port table, since a profile's port can be overridden per-host.
- For every catalog profile, it independently probes that profile's own `/v1/models` endpoint. **A listening port is never trusted on its own** — a real, previously-measured failure mode is an unrelated service occupying the same port and answering something that merely looks like a 200; only a genuine, parseable OpenAI-shaped model listing counts as "running."
- Only profiles that pass that live probe become aliases. A profile that stops answering disappears from the alias set on the very next sync — there is no stale/cached state.

Naming is always namespaced: `llmctl-<profile>` for Claude Code, `kimi-llmctl-<profile>` for Kimi Code — never a bare profile name, so an llmctl-backed alias is always visually distinct from a native account or any other provider alias.

2. Switching

Launching an `llmctl-<profile>` alias that is **not** the currently-active llmctl profile triggers an automatic, on-demand switch (`_cma_llmctl_ensure_active()` in `scripts/lib.sh`, shared by the Claude, Kimi, and Pi launch wrappers):

1. It checks `llmctl status` first — if the requested profile is *already* the sole running one, nothing else happens (no needless switch latency on repeated use of the same alias).
2. Otherwise it runs `llmctl switch <profile>`, which **stops every other running profile and starts exactly the requested one** — exclusive by design, mirroring llmctl's own `switch` semantics. Only one llmctl-backed alias is ever considered active at a time.
3. If the target profile doesn't fit the host's current resources, llmctl refuses the switch and the launch aborts with that reason — the previously-active profile is independently **re-verified** as still live afterward, rather than assumed so merely because the switch command's exit code said so.
4. llmctl's own switch has a documented, real failure mode: its automatic rollback-to-the-previous-state is *best-effort*, and can itself fail. When that happens, claude_toolkit surfaces it as a **distinct, higher-severity `CRITICAL: llmctl rollback also failed`** message — never silently conflated with an ordinary refusal — so you know the host may now have fewer services running than before, not just that one switch didn't happen.

Switching, or an llmctl-backed alias disappearing, never affects any native account alias or any other provider alias — they are completely independent.

3. LAN-exposure warning

By default, llmctl binds its profiles to `0.0.0.0` — reachable from other machines on your local network, not just this one (llmctl's own documented trade-off: its OpenAI-compatible API has no built-in authentication). llmctl itself reports no bind-address field in any of its own output, so claude_toolkit determines this independently: it reads the **real kernel listening-socket state** for the profile's resolved port (via `ss`) the moment the alias becomes available, and surfaces a plain warning naming the exposure if the bind address isn't loopback-only. The alias remains fully usable either way — this is information, not a block.

If you want a profile to be loopback-only, that's an llmctl-side setting, not a claude_toolkit one: set `LLMCTL_BIND_HOST=127.0.0.1` globally, or `LLMCTL_BIND_HOST_<PROFILE>=127.0.0.1` for just one profile, before starting it with llmctl.

4. Context-size warning

An llmctl-backed alias's advertised context is read from the profile's own real, currently-configured value (its `/v1/models` response's `meta.n_ctx` field, or failing that llmctl's own catalog `ctx` field) — never a flat, one-size-fits-all number. A CLI agent's own turn carries real overhead before your prompt ever gets added (system prompt + tool schemas), and some llmctl profiles are deliberately configured with a small context window that cannot hold that overhead. When a profile's real context can't clear that floor, you'll see an explicit warning at the point the alias appears, naming the real context and the shortfall — the alias is still created and usable, but a real agentic turn against it may fail with a context-exceeded error from the model itself.

This is a property of how *that specific llmctl profile* was configured, not a claude_toolkit defect — the fix, if you want one, is either using a differently-configured (larger-context) profile for agentic work, or using the small-context profile directly via its own API for simpler, non-agentic use where the smaller window is fine.

5. Troubleshooting

- **A profile I started doesn't show up as an alias.** Confirm it's actually answering: `llmctl status` should list it as running. Then re-sync: `claude-providers sync`. Detection only ever reflects profiles that genuinely answer their own `/v1/models` endpoint right now — a profile that is merely *listed* in llmctl's catalog but not currently serving will never produce an alias.

- **Switching seems to hang or fail.** The underlying `llmctl switch` call can take real time (stopping one model, starting another) — this is expected, not a toolkit bug. If it genuinely fails, the launch aborts with llmctl's own reason printed; that reason (resource fit, a crashed engine, etc.) is the thing to address, not claude_toolkit's wrapper around it.
- **llmctl isn't installed on this host at all.** Everything behaves exactly as if llmctl were simply absent — zero llmctl-related aliases, no error, no dependency introduced. You can use every other part of claude_toolkit normally.

5a. Model validation gate (§11.4.277) and Vision QA (§11.4.278)

Local models are only as trustworthy as their last known-answer check. Constitution **§11.4.277** puts that check inside llmctl, not inside this toolkit:

- `llmctl validate <profile>` runs deterministic known-answer suites (chat, coder, embedding, and the Vision suite `vision-golden-v1`). Its exit codes (llmctl `docs/validation.md`): **0** every check passed, **1** any wrong result (a defective model/runtime), **2** could not determine (not served, wrong model served, truncated output, pressure refusal, ...; UNDETERMINED is never a pass).
- `llmctl require-validated <profile>`: **0** only for a fresh, signed PASS record that matches what would run now; **1** not validated; **2** cannot determine. **75** and **98** are not validation verdicts: 75 is the host-pressure gate refusing to start a unit (it is also the code of a failed rollback, which is why the toolkit tells them apart by re-probing the running set), and 98 is the port-in-use skip of the systemd/onnx path.
- `LLMCTL_VALIDATE_STRICT=1` makes an UNDETERMINED result stop the profile; llmctl treats consumer-facing profiles strictly unless `LLMCTL_VALIDATE_STRICT=0` is set, so set it to `1` explicitly for decision-bearing use.
- llmctl runs the gate on `start`, `switch`, `auto`, `enable` and `restart`. Launching an `llmctl-<profile>` alias runs `llmctl switch <profile>` (§2) **only when that profile is not already the sole running one**, and then the gate runs inside llmctl; the toolkit forwards your environment unchanged and any non-zero exit from `switch` aborts the launch with llmctl's own message and exit code. When the profile is already running no switch happens and the start-time gate does not run again: under `LLMCTL_VALIDATE_STRICT=1` the toolkit therefore asks `llmctl require-validated <profile>` itself (exit 0 proceeds; anything else aborts the launch with that code). Without `STRICT=1` an already-running profile launches as before.
- A refusal is never turned into a launch against an unvalidated model. A wrong answer is a model/quantisation/`mmproj`/

template defect to investigate, never a reason to retry until green.

Vision QA (§11.4.278). HelixQA reads real screens with a Vision model; that model must hold a fresh `require-validated` PASS first. The toolkit's part is only to provide the `llmctl-vision` alias and keep the gate's environment and refusals intact — proven by `scripts/tests/test_llmctl_vision_gate_wiring.sh` (a fake llmctl; no model is run). The toolkit runs **no** Vision known-answer suite of its own, and a PASS in this repo says nothing about Vision accuracy.

The llmctl units are host-safe by construction (§11.4.277(E): no `Restart=always`, memory limits, refuse-to-start under memory pressure); see llmctl's `docs/host-safety.md`.

Related: llmctl is itself a constitution *hub engine*, consumed by reference and never nested — see the README's hub-engine section and the placement diagram.

6. What this integration does not do

claude_toolkit does not install, configure, or manage llmctl itself, and does not modify llmctl's own configuration or behavior. Where analyzing llmctl's own codebase surfaced genuine defects in that separate project, they are documented — not fixed here, since claude_toolkit does not own that repository — in `../research/2026-10-02-llmctl-upstream-findings.md`.

7. Diagrams

Visual walkthroughs of the two flows described above:

- Detection flow (source) — catalog → per-profile probe → context/LAN honesty checks → alias.
- Hub-engine placement (source) — where llmctl and claude_toolkit sit in a consuming project, and how dependencies resolve without nesting.
- Switch flow (source) — on-demand switch, the post-failure re-probe, and the `CRITICAL: rollback also failed` path.